CS-781 Capstone, Phase 6: Abstractive Legal Summarization

Fine-tuning the four Phase-1 language models as summarizers

Manish Kumar (251330604)
CS-781, IIT Kanpur

November 2026

Abstract

We generate abstractive summaries of Indian court judgments with the four legal language models pre-trained in Phase 1. The decoder-only models (A1, A3, A4) read a condensed source followed by the summary; the encoder–decoder (A2) reads four source chunks with fusion-in-decoder. Nine first-round experiments cover all four architectures and eight adaptation techniques (full fine-tuning, LoRA, LoRMA, QLoRA, AdaLoRA, Houlsby adapters, prefix and prompt tuning); a 6-block A4 student is distilled from the full A4 model. On a 300-document validation split, controlling repetition at decoding time adds +0.0033 to +0.0081 BERTScore F1, and longer sources (up to 4,096–8,192 tokens, by position interpolation) give the best single model (A1, 4,096 tokens: 0.8554). Picking, per document, the candidate summary that agrees most with the others (minimum-Bayes-risk selection over 12 candidates) reaches 0.8577. All numbers are generated from the run outputs by `p6/report/make_figures.py`.

Contents

1 Introduction **3**

2 Data **3**

3 Methods **3**
3.1 Summarizer on the four architectures . . . 3
3.2 Adaptation techniques . . . 4
3.3 Training . . . 4
3.4 Combining runs . . . 5
3.5 Submissions . . . 5

4 Results **5**
4.1 All experiments . . . 5
4.2 Source length . . . 6
4.3 Decoding: repetition, length, beams, sampling . . . 7
4.4 Knowledge distillation and efficiency . . . 7
4.5 Combining runs . . . 8
4.6 Per-document scores . . . 8
4.7 Submission files . . . 9

5 Analysis **10**

6 Limitations and next steps **10**

7 Conclusions **10**

8 Reproducibility **11**

1 Introduction

The task is to write an abstractive summary of a long legal judgment. The primary metric is BERTScore F1 ("legal BERTScore"; the encoder used by the organisers is not stated), with ROUGE-1 F1 and ROUGE-L F1 as secondary metrics. The rules require the Phase-1 models as the only pretrained weights (no other pretrained model, external corpora or LLM APIs), all four architectures, at least six substantially different experiments with at least five adaptation techniques, LoRMA and a knowledge-distillation experiment.

Local metric. We score with `bert_score` using its English default encoder (roberta-large), only as an evaluation tool (`p6/summ/metrics.py`). It truncates both texts at 512 encoder tokens, so only about the first 400 words of a summary and of the reference are compared; ROUGE sees the whole text.

2 Data

Table 1: Splits (lengths in words). Compression = document length / summary length.

Split	Documents	Doc. median	Doc. 90th pct.	Summary median	Summary 90th pct.	Compression
train	6,703	3,123	7,843	640	1,507	4.6
val	300	3,246	8,109	612	1,418	4.9
test	100	3,171	8,563	–	–	–

There is no public development set. The training file has 7,030 entries and 7,003 distinct documents (exact duplicates dropped, whitespace-normalised hash); 300 of them are drawn once with a fixed seed as the validation split, leaving 6,703 for training. Reference summaries have a median of 612 words and 75% are longer than 400 words, i.e. longer than what BERTScore compares.

Table 2: Reference points on the validation split (no model).

Summary	Words	BERTScore F1	ROUGE-1	ROUGE-L
another document's reference summary	827	0.8023	0.321	0.161
first 400 words of the document	400	0.8297	0.429	0.212
evenly spread sentences, 450 words	426	0.8410	0.485	0.253

3 Methods

3.1 Summarizer on the four architectures

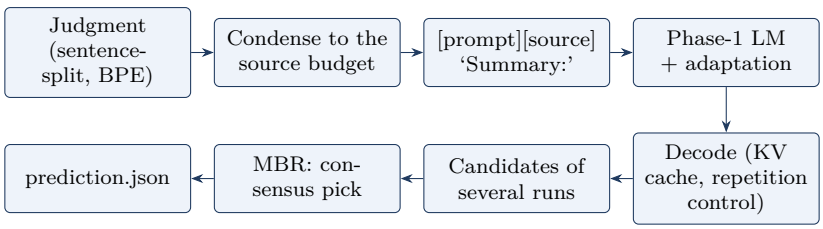


Figure 1: Pipeline (`p6/summ/`, `train.py`, `mbr.py`).

- **Layout (A1, A3, A4).** One sequence: a short prompt, the condensed source (left-padded), the separator " `Summary:`", then the summary and end-of-sequence. The loss is computed on the summary tokens only (up to 768 tokens; the median reference is about 770 tokens). A3

attends bidirectionally over the prompt and source (its prefix-LM design) and causally over the summary.

- **A2 (encoder–decoder)** uses fusion-in-decoder [7]: the source is split into 4 chunks of 1,024 tokens, each encoded separately, and the decoder attends to all of them.
- **Long sources.** Judgments have a median of about 3,100 words, more than the 1,024-token Phase-1 context. A1's learned position table is linearly interpolated to the needed length [2] (re-interpolated when a run starts from a shorter-source run, `init_run`); A3 (ALiBi) and A4 (RoPE) extrapolate without changes. Budgets: 1,280 tokens (first round), 2,560, 4,096, 6,144 and 8,192 (later rounds).
- **Condensation** (`summ/data.py`) when a document exceeds the budget: *salient* (default) keeps the opening of the case (15% of the budget; 30% in S7c) plus the sentences closest to the document's TF-IDF centroid, in document order; *spread* (S2) keeps evenly spaced sentences.
- **Decoding**: greedy with a KV cache, 300–800 new tokens. The first decode of the first round used no repetition penalty and blocked repeated 4-grams; all later decodes use repetition penalty 1.15 [8] and block repeated 3-grams. Beam search (4 beams) and sampling ($T = 0.7$) were also tried.

3.2 Adaptation techniques

- **Full fine-tuning**: F1 (A4), F2 (A2), L2 (A1), L3 (A3) and all long-source runs S*.
- **LoRA** [6] (E2, S2; A1): $r = 32$, $\alpha = 64$.
- **LoRMA** [1] (E1; A4): $W = (I + \frac{\alpha}{r}BA)W_0$, $r = 32$, $\alpha = 64$.
- **QLoRA** [3] (E4; A2 + FiD): frozen weights in 4-bit NF4, LoRA $r = 32$.
- **AdaLoRA** [12] (E6; A1): SVD-form update, rank pruned from 32 to 16.
- **Houlsby adapters** [5] (E3; A3): bottleneck 128.
- **Prefix tuning** [11] (E5; A3): 64 learned key/value positions per layer from a 512-wide MLP, implemented as a pre-filled KV cache.
- **Prompt tuning** [10] (E7; A4): 64 learned input embeddings.
- **Knowledge distillation** [4] (K1): the student is A4 with 6 of its 10 blocks (0, 2, 4, 5, 8, 9; global-attention layers kept), fully fine-tuned with $0.5\,\mathrm{CE} + 0.5\,T^2\,\mathrm{KL}(p^T_{\text{teacher}} \,\|\, p^T_{\text{student}})$ per summary token, $T = 2$; the teacher is F1 (full A4).

3.3 Training

AdamW, 16 sequences per optimizer step (micro-batches of 1–4 with gradient accumulation), linear warm-up (100 steps; 50 for runs that continue another run) then cosine decay, weight decay 0.01, dropout 0.1, bf16. First round: 4 epochs, learning rate 10^{-4} for full fine-tuning, 5×10^{-4} for LoRA, LoRMA, QLoRA and AdaLoRA, 10^{-3} for adapters and prefix tuning, 3×10^{-3} for prompt tuning. Later full fine-tuning runs: 5–6 epochs, or 2–3 epochs at 4–5×10^{-5} when continuing a shorter-source run (marked * in Table 3: S4 from S3, S7 from S6, S8 from S7, S9 from S5, S3b from S1). The state with the lowest validation loss is kept; it then decodes the validation and test documents.

3.4 Combining runs

- **Token-level ensemble** (`ensemble.py`): several models decode together, averaging their next-token log-probabilities.
- **Minimum-Bayes-risk (MBR) selection** [9] (`mbr.py`): for each document, every run's summary is a candidate; the candidate with the highest mean agreement (ROUGE-1 F1 + ROUGE-2 F1) with the other candidates is submitted. No extra model is involved.

3.5 Submissions

`prediction.json` is a list of `{"id", "summary"}` for the 100 test documents, at the root of a zip.

4 Results

4.1 All experiments

Table 3: Validation results (300 documents). First round decoded with repetition control ("rp"). Ep. = epochs (* continues a shorter-source run). Loss = validation loss on summary tokens.

Run	Arch	Method	Source	Ep.	Trainable	Loss	BERTScore F1	ROUGE-1	ROUGE-L	Words
F1	A4	full fine-tuning	1,280	4	87.51M	2.056	0.8486	0.495	0.244	414
F2	A2	full fine-tuning + FiD	4×1024	4	89.42M	2.727	0.8329	0.421	0.203	371
E1	A4	LoRMA	1,280	4	4.42M	2.088	0.8480	0.499	0.245	424
E2	A1	LoRA	1,280	4	5.14M	2.133	0.8493	0.526	0.260	472
E3	A3	Houlsby adapters	1,280	4	3.19M	2.175	0.8481	0.508	0.244	439
E4	A2	QLoRA (NF4) + FiD	4×1024	4	5.41M	2.816	0.8315	0.407	0.199	349
E5	A3	prefix tuning	1,280	4	6.75M	2.336	0.8382	0.482	0.215	568
E6	A1	AdaLoRA	1,280	4	5.14M	2.205	0.8463	0.521	0.248	519
E7	A4	prompt tuning	1,280	4	0.05M	2.411	0.8336	0.458	0.207	582
L2	A1	full fine-tuning	1,280	6	82.88M	2.066	0.8505	0.524	0.267	455
L3	A3	full fine-tuning	1,280	6	81.28M	2.096	0.8484	0.501	0.246	417
K1	A4	KD student, 6 blocks (full fine-tuning)	1,280	6	62.33M	2.258	0.8479	0.503	0.246	418
S2	A1	LoRA	1,280	4	5.14M	2.145	0.8452	0.505	0.246	471
S1	A4	full fine-tuning	2,560	6	87.51M	1.752	0.8519	0.509	0.262	410
S3	A4	full fine-tuning	4,096	5	87.51M	1.593	0.8518	0.520	0.265	427
S3b	A4	full fine-tuning	4,096	3*	87.51M	1.593	0.8528	0.517	0.266	413
S4	A4	full fine-tuning	8,192	3*	87.51M	1.536	0.8515	0.517	0.267	419
S5	A3	full fine-tuning	4,096	5	81.28M	1.595	0.8523	0.510	0.262	409
S9	A3	full fine-tuning	6,144	2*	81.28M	1.476	0.8519	0.501	0.264	398
S6	A1	full fine-tuning	2,560	6	83.86M	1.732	0.8520	0.540	0.282	502
S7	A1	full fine-tuning	4,096	3*	85.04M	1.539	0.8544	0.513	0.273	432
S7b	A1	full fine-tuning	4,096	3*	85.04M	1.530	0.8543	0.505	0.273	402
S7c	A1	full fine-tuning	4,096	3*	85.04M	1.519	0.8554	0.518	0.280	427
S8	A1	full fine-tuning	8,192	2*	88.19M	1.453	0.8530	0.506	0.271	418

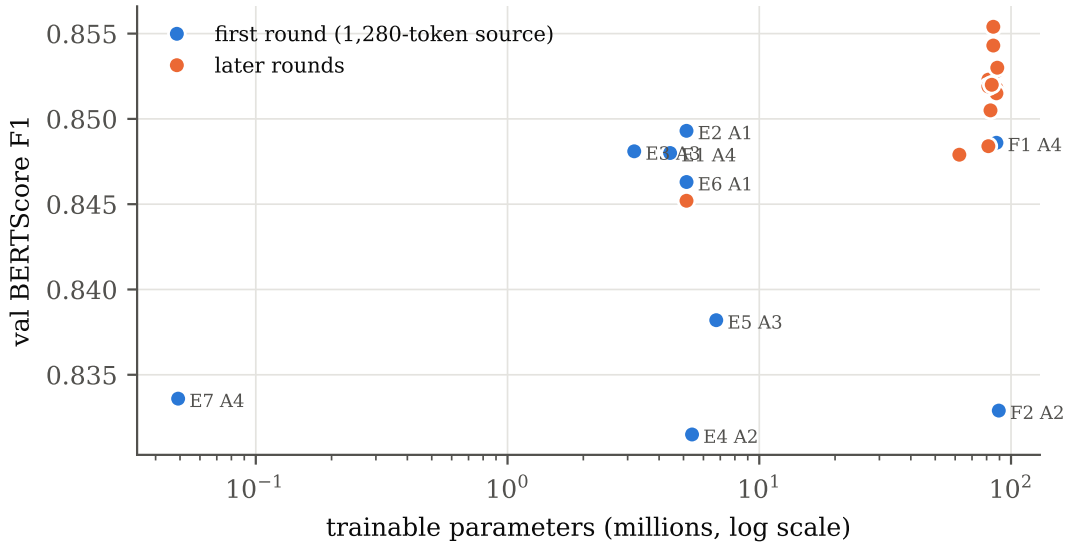


Figure 2: Validation BERTScore F1 against the number of trained parameters.

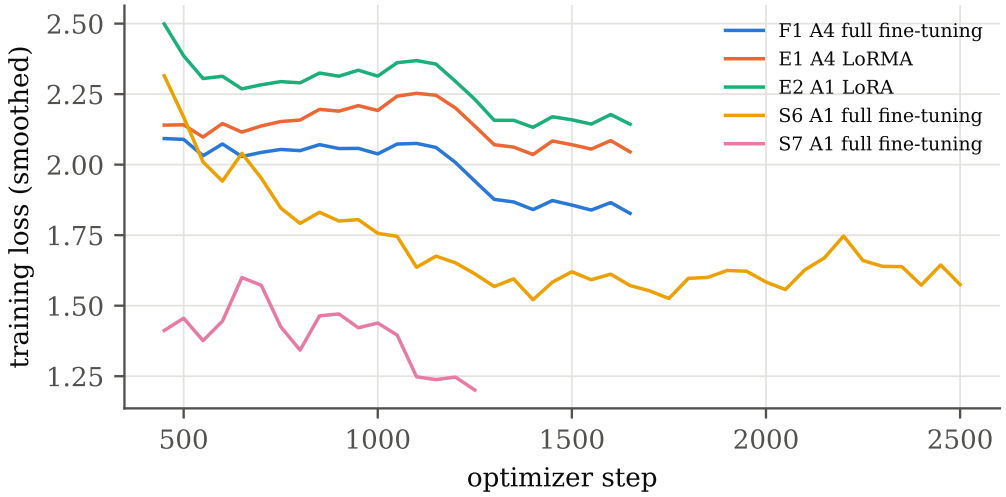


Figure 3: Training loss of selected runs (moving average).

4.2 Source length

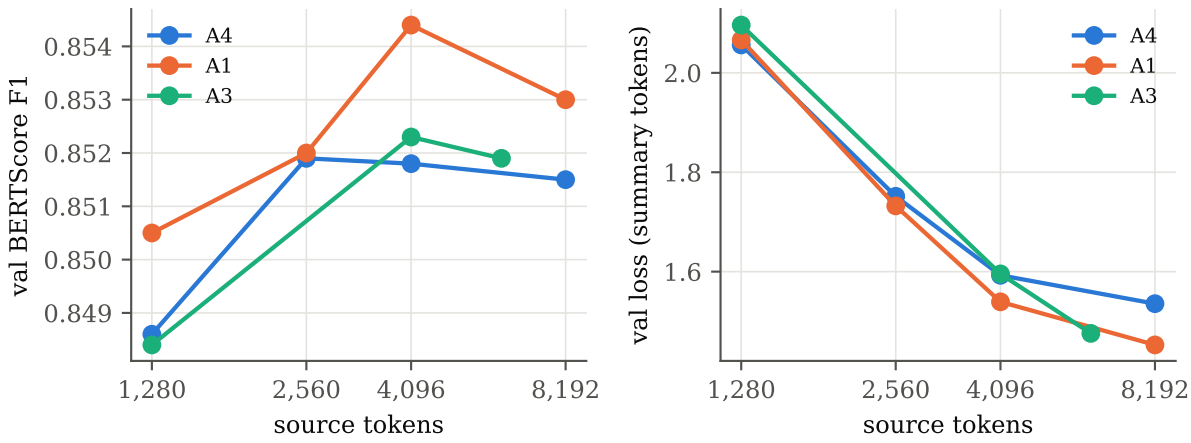


Figure 4: Full fine-tuning with growing source budgets, per architecture (1,280-token points: F1, L2, L3).

4.3 Decoding: repetition, length, beams, sampling

Table 4: First-round runs, first decode (penalty 1.0, no repeated 4-grams) vs. repetition control (penalty 1.15, no repeated 3-grams). Distinct = share of distinct word trigrams per summary.

			BERTScore F1		ROUGE-1		Distinct 3-grams		Words	
Run	Arch	Method	first	rp	first	rp	first	rp	first	rp
F1	A4	full fine-tuning	0.8434	0.8486	0.477	0.495	0.915	1.000	436	414
F2	A2	full fine-tuning + FiD	0.8251	0.8329	0.394	0.421	0.860	0.999	413	371
E1	A4	LoRMA	0.8423	0.8480	0.473	0.499	0.907	1.000	462	424
E2	A1	LoRA	0.8444	0.8493	0.504	0.526	0.918	1.000	492	472
E3	A3	Houlsby adapters	0.8430	0.8481	0.493	0.508	0.902	1.000	490	439
E4	A2	QLoRA (NF4) + FiD	0.8234	0.8315	0.383	0.407	0.858	0.999	393	349
E5	A3	prefix tuning	0.8315	0.8382	0.441	0.482	0.866	0.999	593	568
E6	A1	AdaLoRA	0.8430	0.8463	0.506	0.521	0.905	1.000	572	519
E7	A4	prompt tuning	0.8275	0.8336	0.419	0.458	0.872	0.999	617	582

Table 5: Decoding variants: new-token range, repetition penalty, blocked n-gram size, search, number of validation documents scored (not all variants were scored on all 300).

Run	New tokens	Pen.	No-rep.	Search	Docs	Words	BS-F1	BS-P	BS-R	ROUGE-1	ROUGE-L
F1	300–800	1.0	4	greedy	300	436	0.8434	0.8486	0.8384	0.477	0.251
F1	200–600	1.0	4	greedy	150	377	0.8440	0.8510	0.8374	0.471	0.253
F1	500–1000	1.0	4	greedy	150	505	0.8421	0.8444	0.8400	0.494	0.255
F1	300–800	1.15	3	greedy	300	414	0.8486	0.8512	0.8461	0.495	0.244
S3	300–800	1.15	3	greedy	300	427	0.8518	0.8527	0.8511	0.520	0.265
S3	450–900	1.15	3	greedy	300	484	0.8515	0.8495	0.8537	0.540	0.266
S3	300–800	1.15	3	beam 4	100	421	0.8522	0.8512	0.8534	0.525	0.275
S7	300–800	1.15	3	greedy	300	432	0.8544	0.8532	0.8557	0.513	0.273
S7	300–800	1.15	3	beam 4	300	423	0.8536	0.8520	0.8554	0.507	0.271
S7	300–800	1.15	3	sample T=0.7	300	424	0.8496	0.8484	0.8509	0.502	0.255
S7	300–800	1.15	3	sample T=0.7	300	428	0.8501	0.8484	0.8519	0.505	0.258

4.4 Knowledge distillation and efficiency

Table 6: Size, compute and memory. Decode = seconds per validation document (batched).

Run	Arch	Method	Source	Total	Trainable	Train (min)	Decode (s/doc)	Peak GB
F1	A4	full fine-tuning	1,280	87.5M	87.51M	30	0.61	6.6
K1	A4	KD student, 6 blocks (full fine-tuning)	1,280	62.3M	62.33M	58	0.81	6.3
E1	A4	LoRMA	1,280	91.9M	4.42M	32	0.67	5.9
E2	A1	LoRA	1,280	86.4M	5.14M	26	0.64	4.7
L2	A1	full fine-tuning	1,280	82.9M	82.88M	43	0.59	5.1
S7	A1	full fine-tuning	4,096	85.0M	85.04M	55	1.44	3.4
S4	A4	full fine-tuning	8,192	87.5M	87.51M	136	3.46	7.2
S9	A3	full fine-tuning	6,144	81.3M	81.28M	69	2.46	14.8

The KD student K1 has 62.3M parameters against 87.5M for its teacher F1, and scores 0.8479 BERTScore F1 against 0.8486 for the teacher (same decoding).

4.5 Combining runs

Table 7: MBR selection and the token-level ensemble (validation).

Name	Candidates	Docs	BERTScore F1	ROUGE-1	ROUGE-L	Words
MBR1_top4	4	300	0.8498	0.531	0.260	483
MBR2_long	5	300	0.8518	0.530	0.268	448
MBR4_s7samp	5	300	0.8539	0.525	0.273	453
MBR5_s7samp_beam	9	300	0.8559	0.528	0.283	440
MBR6_long8	8	300	0.8565	0.537	0.287	452
MBR7_all12	12	300	0.8577	0.535	0.289	439
ENS_t3 (token-level ensemble)	3	100	0.8483	0.511	0.259	449

Table 8: MBR7: share of validation documents for which each candidate run was picked (%).

Candidate	Picked (%)
S7_a1_full_src4096	26.0
S7b_a1_full_src4096_seed1	18.0
S8_a1_full_src8192	13.3
S7c_a1_full_src4096_head30	13.3
S7_a1_full_src4096:beam4	13.0
S6_a1_full_src2560	6.0
S1_a4_full_src2560	3.0
S4_a4_full_src8192	2.7
S5_a3_full_src4096	1.7
S3b_a4_full_src4096_fromS1	1.7
S3_a4_full_src4096	1.0
S9_a3_full_src6144	0.3

4.6 Per-document scores

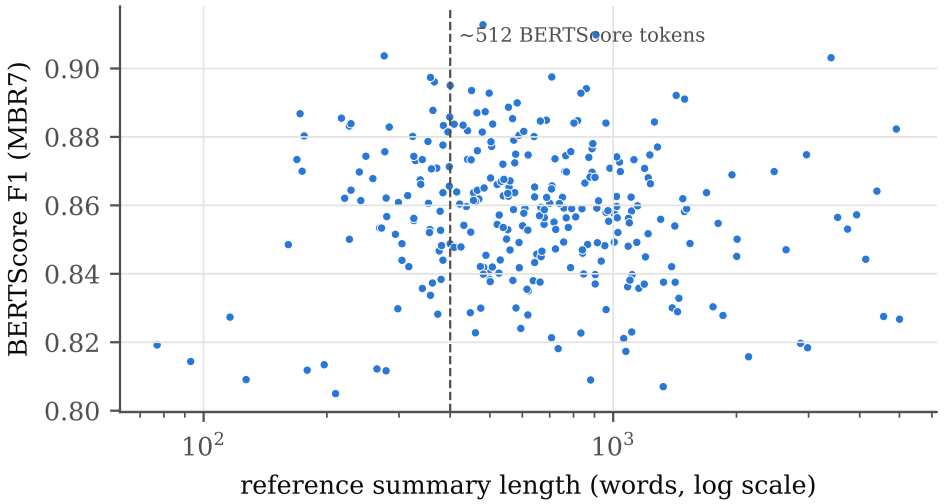


Figure 5: MBR7 BERTScore F1 per validation document against the reference length. Correlation with reference length -0.10; with log document length -0.40.

Table 9: Mean BERTScore F1 by reference-length quartile (words). MBR7 beats S7 on 45% of documents.

Quartile	Reference words	Docs	MBR7	S7
Q1	77–406	75	0.8585	0.8552
Q2	406–611	75	0.8617	0.8568
Q3	611–961	74	0.8585	0.8539
Q4	961–5002	76	0.8522	0.8517

4.7 Submission files

51 format-checked zips are in `p6/submissions/`. Uploaded first: `MBR7_all12` (0.8577, ROUGE-1 0.535, ROUGE-L 0.289), `MBR6_long8` (0.8565), the best single models S7 and S7c, and E1 (LoRMA) and K1 (KD) for technique coverage.

To be written by the author — leaderboard scores

Add the Codabench test BERTScore F1, ROUGE-1 and ROUGE-L of each uploaded submission and say which six or more count towards the requirement.

File
E1_a4_lorma_20261029_011558.zip
E1_a4_lorma_rp_20261029_125153.zip
E2_a1_lora_20261029_010844.zip
E2_a1_lora_beam4_20261029_125551.zip
E2_a1_lora_rp_20261029_122634.zip
E3_a3_adapter_20261029_013335.zip
E3_a3_adapter_rp_20261029_124149.zip
E4_a2_qlora_fid_20261029_014330.zip
E4_a2_qlora_fid_rp_20261029_131154.zip
E5_a3_prefix_20261029_014937.zip
E5_a3_prefix_rp_20261029_130409.zip
E6_a1_adalora_20261029_020936.zip
E6_a1_adalora_rp_20261029_123429.zip
E7_a4_prompt_20261029_022559.zip
E7_a4_prompt_rp_20261029_132221.zip
F1_a4_full_20261029_011153.zip
F1_a4_full_20261029_011808.zip
F1_a4_full_20261029_012130.zip
F1_a4_full_20261029_012546.zip
F1_a4_full_rp_20261029_123326.zip
F2_a2_full_fid_20261029_011027.zip
F2_a2_full_fid_rp_20261029_125645.zip
K1_kd_a4_6l_20261029_132458.zip
L2_a1_full_ep6_20261029_130804.zip
L3_a3_full_ep6_20261029_142255.zip
MBR1_top4_20261029_124342.zip
MBR2_long_20261029_234120.zip
MBR3_long3_20261030_012011.zip
MBR4_s7samp_20261030_151519.zip
MBR5_s7samp_beam_20261030_151615.zip
MBR6_long8_20261030_151937.zip
MBR7_all12_20261103_173301.zip
S1_a4_full_src2560_20261029_145723.zip
S2_a1_lora_spread_20261029_134815.zip
S3_a4_full_src4096_20261029_152200.zip
S3_a4_full_src4096_beam4_20261030_003407.zip
S3_a4_full_src4096_long_20261030_000715.zip
S3b_a4_full_src4096_fromS1_20261103_172719.zip
S4_a4_full_src8192_20261030_021722.zip
S5_a3_full_src4096_20261030_020603.zip
S6_a1_full_src2560_20261030_011744.zip
S7_a1_full_src4096_20261030_022436.zip
S7_a1_full_src4096_beam4_20261030_024521.zip
S7_a1_full_src4096_samp1_20261030_151329.zip
S7_a1_full_src4096_samp2_20261030_151329.zip
S7_a1_full_src4096_samp3_20261030_151329.zip
S7_a1_full_src4096_samp4_20261030_151329.zip
S7b_a1_full_src4096_seed1_20261103_170900.zip

File
S7c_a1_full_src4096_head30_20261103_170900.zip
S8_a1_full_src8192_20261030_031609.zip
S9_a3_full_src6144_20261103_171510.zip

5 Analysis

To be written by the author — analysis of the results

The course rules do not allow LLM-written result analysis. Questions the course asks (§12.5), with the evidence above:

- Which adaptation techniques are effective for legal summarization, and can parameter-efficient methods retain the quality of full adaptation (Table 3, Figure 2)?
- How do long-document strategies affect quality (condensation: E2 vs. S2; source length: Figure 4; FiD for A2)?
- Do the summaries preserve important legal facts and reasoning? (Read a sample of validation summaries next to their references.)
- How well do they cover important information (BERTScore recall, ROUGE)?
- How much redundancy and repetition is there (Table 4)?
- How does generation length affect BERTScore and ROUGE (Table 5, Table 9)?
- Does domain adaptation (Phase-1 pre-training on legal text) help?
- Does distillation preserve quality while reducing size and compute (Table 6)?
- How do model size, memory and inference cost change across experiments?
- Are the Phase-1 representations useful for abstractive summarization (compare with Table 2)?

6 Limitations and next steps

Facts that limit the conclusions (the interpretation is the author's):

- The local BERTScore encoder (roberta-large) is probably not the organisers' "legal BERTScore", and it compares only about the first 400 words.
- Validation has 300 documents; differences of about 0.001 BERTScore are within noise. One seed per run (S7b repeats S7 with another seed: 0.8543 vs. 0.8544).
- The MBR candidate pool was chosen on the same validation split it is scored on.
- Some decoding variants were scored on 100 or 150 documents only (Table 5).
- The 1,024-token Phase-1 models were run at up to 8,192 tokens without long-context pre-training.

To be written by the author — what you would do next

With more time or compute: ...

7 Conclusions

To be written by the author — conclusions

Summarise what you conclude from the experiments, in your own words.

8 Reproducibility

Token cache	built on first use by `summ/data.py` (`build_cache`)
Experiments	`bash run_queue.sh f1_a4_full e1_a4_lorma ...` (one config each, logs in `logs/`)
One experiment	`python train.py -config configs/s7_a1_full_src4096.yaml`
Re-decode	`bash redecode.sh rp "gen.repetition_penalty=1.15 gen.no_repeat_ngram=3" f1_a4_full ...`
MBR zip	`python mbr.py -name MBR7_all12 -cand S7_a1_full_src4096 S7_a1_full_src4096:beam4 ...`
Token-level ensemble	`python ensemble.py -name ENS_t3 -runs E2_a1_lora F1_a4_full E3_a3_adapter`
Figures and tables	`python p6/report/make_figures.py`

To be written by the author — repository link

GitHub: `https://github.com/...`

LLM declaration

To be written by the author — LLM declaration (mandatory)

Fill in the course questionnaire. Describe honestly which parts used an LLM assistant (for example the data pipeline, model and training code, figures and this report's methods/results text) and which parts are your own (design decisions, hyperparameters, analysis, conclusions).

References

[1] H. Bihany, S. Patel, A. Modi. LoRMA: Low-rank multiplicative adaptation for LLMs. Findings of ACL 2025.

[2] S. Chen et al. Extending context window of large language models via positional interpolation. arXiv:2306.15595, 2023.

[3] T. Dettmers et al. QLoRA: Efficient finetuning of quantized LLMs. NeurIPS 2023.

[4] G. Hinton, O. Vinyals, J. Dean. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.

[5] N. Houlsby et al. Parameter-efficient transfer learning for NLP. ICML 2019.

[6] E. J. Hu et al. LoRA: Low-rank adaptation of large language models. ICLR 2022.

[7] G. Izacard, E. Grave. Leveraging passage retrieval with generative models for open domain question answering. EACL 2021.

[8] N. S. Keskar et al. CTRL: A conditional transformer language model for controllable generation. arXiv:1909.05858, 2019.

[9] S. Kumar, W. Byrne. Minimum Bayes-risk decoding for statistical machine translation. HLT-NAACL 2004.

[10] B. Lester, R. Al-Rfou, N. Constant. The power of scale for parameter-efficient prompt tuning. EMNLP 2021.

[11] X. L. Li, P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. ACL 2021.

[12] Q. Zhang et al. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. ICLR 2023.